

# NexusLLM RAG Upgrade & AI-V vs AI-D Postponement Plan

Engineering priority plan — stabilize and upgrade NexusLLM first; keep the new neural-conflict project on the roadmap.

## 1. Strategic Decision

**Decision:** Postpone the AI-V vs AI-D project for now. Do not delete the concept. Preserve it as a future experimental / multi-agent AI portfolio project.

The immediate priority is **NexusLLM**, especially the RAG subsystem and backend reliability. This keeps the existing project moving toward a more defensible, production-style AI system instead of splitting effort across multiple major builds.

| Priority | Project / Area         | Status             | Action                                   |
|----------|------------------------|--------------------|------------------------------------------|
| P0       | NexusLLM backend       | Needs attention    | Diagnose connection failure first        |
| P0       | NexusLLM RAG           | Upgrade target     | Improve retrieval, grounding, evaluation |
| P1       | NexusLLM observability | Existing direction | Strengthen traces and failure visibility |
| P2       | AI-V vs AI-D           | Postponed          | Keep specification; build later          |

## 2. Current NexusLLM Problem

The supplied NexusLLM screenshot shows the Engine interface with routing modes **AUTO, DIRECT, RAG, and AGENTIC**. The interface currently reports: “An error occurred connecting to the backend engine.”

This should be treated as a backend/integration reliability issue before adding more UI features. The routing selector can exist visually, but each mode needs a verified backend execution path and a clear execution trace.

## 3. RAG Upgrade Goal

Upgrade NexusLLM from a basic document-retrieval feature into a measurable, reliable RAG pipeline that can demonstrate retrieval quality, grounded generation, citation coverage, and failure handling.

### Target RAG pipeline

**Ingestion → Parsing → Chunking → Metadata → Embeddings → Vector Index → Hybrid Retrieval → Reranking → Context Assembly → LLM Generation → Grounding / Citation Verification → Response + Trace**

## 4. RAG Upgrade Workstreams

**A. Ingestion** — Support reliable ingestion of PDFs/text and preserve document metadata such as source, page, section, chunk ID, and timestamps.

**B. Chunking** — Test chunk size and overlap rather than assuming one fixed configuration. Preserve headings and semantic boundaries where possible.

**C. Retrieval** — Use hybrid dense + sparse retrieval so semantic similarity and exact terminology can complement each other.

**D. Reranking** — Add a reranking stage after initial retrieval to improve the relevance of the final context passed to the generator.

**E. Context assembly** — Deduplicate overlapping chunks, enforce context limits, and maintain source attribution.

**F. Grounding** — Require answers to be supported by retrieved evidence. If evidence is insufficient, return an explicit uncertainty / no-answer state.

**G. Citations** — Expose document/page/chunk references so the user can inspect where an answer came from.

**H. Evaluation** — Create a fixed evaluation set and measure retrieval and answer quality rather than judging RAG only by manual demos.

**I. Observability** — Trace query → retrieval → reranking → generation → verification, including latency and failure states.

## 5. Recommended RAG Evaluation

| Layer      | Metrics / Checks                  | Purpose                                   |
|------------|-----------------------------------|-------------------------------------------|
| Retrieval  | Recall@K, Precision@K, MRR / nDCG | Does the system retrieve useful evidence? |
| Reranking  | Top-K relevance                   | Does reranking improve context quality?   |
| Generation | Faithfulness / groundedness       | Is the answer supported by evidence?      |
| Citation   | Citation coverage / correctness   | Can claims be traced to sources?          |
| System     | Latency, errors, throughput       | Is the pipeline operationally reliable?   |

## 6. NexusLLM Backend Recovery Plan

- **Step 1 — Reproduce:** Run the backend independently and capture the exact startup error, port, host, and traceback.
- **Step 2 — Health endpoint:** Add a lightweight backend health check so the frontend can distinguish “backend offline” from “query failed.”
- **Step 3 — API contract:** Verify request/response schemas between the frontend and engine.
- **Step 4 — Routing:** Test DIRECT, RAG, and AGENTIC independently before relying on AUTO routing.
- **Step 5 — RAG smoke test:** Upload/index one known document and verify retrieval before invoking the LLM.
- **Step 6 — Execution trace:** Record routing decision, retrieved chunks, reranking, generation, verification, latency, and errors.
- **Step 7 — Failure handling:** Replace generic backend errors with actionable states such as `BACKEND_OFFLINE`, `RETRIEVAL_EMPTY`, `MODEL_ERROR`, or `INVALID_REQUEST`.

## 7. Definition of Done for the RAG Upgrade

- Backend starts reliably and the frontend health check reports `ONLINE`.
- A test document can be ingested and indexed successfully.
- A query retrieves the correct evidence with source metadata.
- Hybrid retrieval can be compared against dense-only retrieval.
- Reranking can be enabled/disabled for controlled evaluation.
- Generated answers expose citations or source references.
- The system can explicitly refuse to answer when evidence is insufficient.
- Execution Trace shows the complete RAG path.
- A small evaluation dataset produces repeatable metrics.
- The README documents architecture, evaluation methodology, and known limitations.

## 8. AI-V vs AI-D — Postponed Roadmap

Keep the concept as a future project. The current design direction is a graph-based multi-agent simulation where AI-V and AI-D are represented as neural/computational systems rather than humanoid characters.

**Future MVP:** environment graph → two agents → state observation → action selection → environment transition → reward/outcome → repeatable experiments.

**Future advanced version:** reinforcement learning, adaptive strategies, 100–1,000+ simulations, statistical analysis, network visualization, experiment tracking, and a cinematic command-center interface.

**Important:** the winner should not be predetermined. The project becomes much more technically meaningful if AI-V and AI-D can win or lose depending on learned policies, environment parameters, and initial conditions.

## 9. Execution Order

1. Fix and verify NexusLLM backend connectivity.
2. Establish health checks and clear backend error states.
3. Stabilize the existing RAG path.
4. Upgrade retrieval with hybrid search.
5. Add reranking and stronger context assembly.

6. Add citation/grounding verification.
7. Build a RAG evaluation dataset and benchmark.
8. Improve execution tracing and observability.
9. Document the upgraded architecture and results.
10. Only after NexusLLM is stable, return to AI-V vs AI-D.

**Portfolio strategy:** NexusLLM remains the primary GenAI systems project. AeroDrift demonstrates MLOps/classical ML. AI-V vs AI-D stays reserved as a later experimental multi-agent/deep-learning project. This prevents project sprawl while increasing the depth of the existing flagship system.

Prepared as an engineering planning document — 25 August 2026